

DATA-STRUCTURES & ALGORITHMS

with

Manoj Prabhakaran

Lecture 19

Sorting (ctd.)

Sorting

- So far:
 - Selection sort and Heapsort:
 - $O(n^2)$ vs $O(n \log n)$. Both in-place, not stable.
 - Insertion sort and Tree sort:
 - $O(n^2)$, in-place vs $O(n \log n)$, not in-place. Both stable.
 - Merge sort
 - $O(n \log n)$. Not in-place. Stable.
- Today: Quicksort

Quicksort

- Another divide-and-conquer sorting algorithm
 - Almost in-place. Not stable.
- Divide into two lists, $S_1 = [\text{items} < \text{pivot}]$ and $S_2 = [\text{items} > \text{pivot}]$
- Combine: $[\text{quicksort}(S_1) \text{ [items == pivot] } \text{quicksort}(S_2)]$
- How is pivot picked? Various approaches
 - Pick the first element: Worst case $O(n^2)$
 - E.g., when the data is sorted or reverse-sorted
 - Pick a random element: Worst case $O(n \log n)$ with high probability
 - Pick the "median of medians": Worst case $O(n \log n)$
 - Deterministically, close enough to the ideal
 - Heuristic choices: Worst case $O(n^2)$, but often better than that
- Hybrid models: switch to another sorting if taking too long

Quicksort: Using Aux Space

```
void qsort (std::vector<int>& A) {  
    int N = A.size();  
    if(N<=1) return;  
    int pivot = A[0]; // may use alternate logic  
    std::vector<int> L, E, G; // Less, Equal and Greater  
    for(int i=0; i<N; ++i) { // scan and populate the 3 arrays  
        if (A[i] < pivot) L.push_back(A[i]);  
        else if (A[i] > pivot) G.push_back(A[i]);  
        else E.push_back(A[i]);  
    }  
    qsort(L); qsort(G);  
    // copy L, E, G to A  
    std::copy(L.begin(), L.end(), A.begin());  
    std::copy(E.begin(), E.end(), A.begin()+L.size());  
    std::copy(G.begin(), G.end(), A.end()-G.size());  
}
```

Quicksort: In-Place Pivoting

```
void qsort (std::vector<int>& A, int left, int right) {  
    if(left>=right) return;  
    int pivot = A[left]; // may use alternate (in-place) logic  
    int L=left, G=right;  
    for(int i=left; i<=G; ) {  
        if (A[i] < pivot) std::swap(A[L++],A[i++]);  
        else if (A[i] > pivot) std::swap(A[G--],A[i]);  
        else ++i;  
    }  
    qsort(A,left,L-1); qsort(A,G+1,right);  
}
```



- Almost in-place: The data itself stays in the original container
 - Recursion uses a stack to store left and right (can't predict them)
 - Recursion depth can be made $O(\log n)$ by the choice of pivot

Merge Sort and Quicksort

- Both are practically important algorithms: Quicksort used in `std::sort` and Merge Sort in `std::stable_sort`
- In practice, combined with several optimisations
 - The base-case is not one element, but a small enough array . Then it switches to, say, insertion sort (in-place and stable).
 - `std::sort`, uses a more complex (deterministic) choice of pivot, and if the recursion depth goes well beyond $\log n$, switches to heapsort
 - `std::stable_sort`, if not enough memory for standard merge sort, falls back to a more complex, but in-place, $O(n \log^2 n)$ -time merge algorithm
 - Heuristics to exploit pre-sorted subsequences and patterns

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

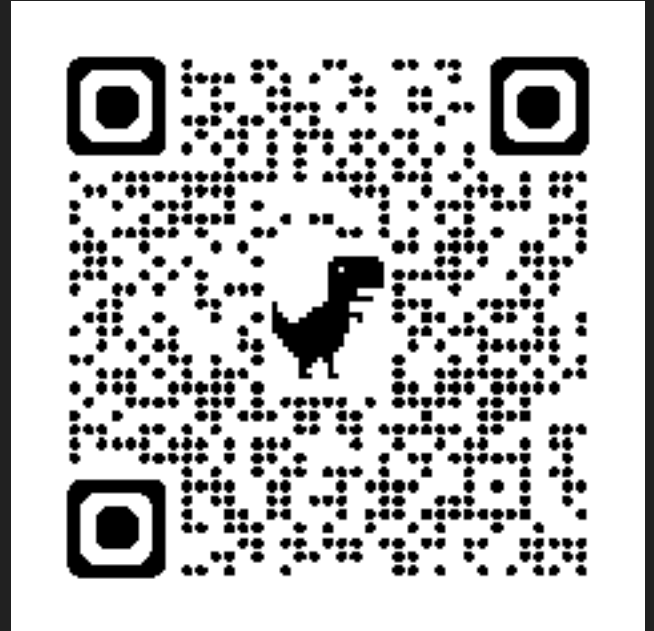
Read the question, choose the answer and submit.

The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

**If you are not in the classroom, a report will
be sent to DADAC!**

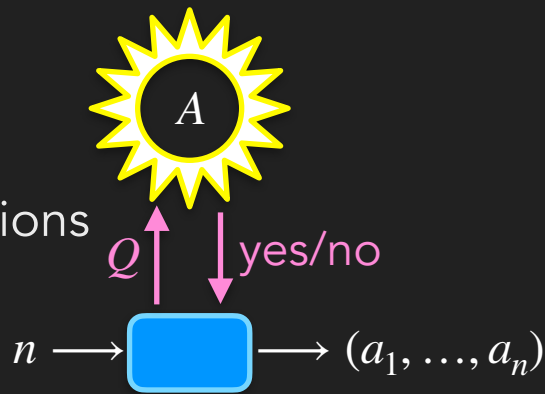


Lower Bound: $\Omega(n \log n)$ Comparisons

- General purpose sorting algorithms are “comparison-based”
 - e.g., `std::sort` and `std::stable_sort` work, by default, on any data-type with `operator<` (and supports moving)
- **Claim:** Any comparison-based sorting algorithm will need to make, in the worst-case, at least $cn \log n$ comparisons, where c is a constant
($\exists n_0, c > 0 \forall n > n_0 \ N_{\text{comp}}(n) \geq cn \log n$)
 - $\Omega(n \log n)$ comparisons
- There are some non-comparison-based sorting algorithms, exploiting additional guarantees/structure on the data
 - E.g., bucket sort and radix sort (can be faster for sorting numbers in certain ranges)

Lower Bound: $\Omega(n \log n)$ Comparisons

- Formalising a comparison-based sorting algorithm:
 - Given n and an “oracle” that answers boolean questions about an n -item vector A (e.g. $A[i] < A[j]$?)
 - To output indices $(a_1, \dots, a_n)$ s.t. $A[a_1] \leq \dots \leq A[a_n]$
- Fix n and a correct sorting algorithm. How many queries should it ask the oracle?
- Key observation:** The final output is completely determined by the response sequence
- Can reproduce the output by running the algorithm and feeding it responses from this sequence (without needing to know A)



Lower Bound: $\Omega(n \log n)$ Comparisons

- Output completely determined by the response sequence
- Suppose the algorithm makes at most k oracle queries
 - W.l.o.g, assume it always makes exactly k queries
 - Because the algorithm can be modified so that it makes additional dummy queries at the end if necessary
- Number of possible response sequences from the oracle is at most 2^k
- So number of distinct outputs is $\leq 2^k$
- But number of distinct outputs by a fully correct algorithm should be $n!$
- $n! \leq 2^k \implies k \geq \log_2(n!) \geq \log_2((n/2)^{n/2})$

